VERIFIED RESEARCH COMPENDIUM

Claude Code Advanced Practitioner's Binder

Strategies, Tools & Verified Claims for Agentic Engineering

Distilled from course analysis, GitHub verification, Anthropic documentation, and multi-source cross-referencing. Every claim tagged with verification status. No single-source dependency.

Compiled by: Prompt AI Solutions (promptaisolutions.com)

Primary Source: "Advanced Claude Code" course (Nyx Drive / YouTube)

Verification Sources: GitHub repos, Anthropic docs, LangChain, Microsoft, academic papers

First Published: April 2, 2026

Last Revised: April 3, 2026

Revision: 1.1

WARNING: This field moves at extraordinary speed. Verify all tool versions, feature availability, and pricing before implementation. Target shelf life: 60-90 days.

REVISION LOG -- This document will age. Check dates before trusting any claim.

Rev	Date	Changes
1.0	2026-04-02	Initial research and claim verification across all modules
1.1	2026-04-03	Expanded expert list, updated star counts, Company Helm found, converted to binder format
1.2	TBD	Re-verify feature flags, pricing, and tool availability (target: May 2026)

How to Use This Binder

For Learning

Work through Modules 1-9 sequentially. Each module is self-contained with verified claims, primary sources, and implementation checklists.

For Reference

Jump to any module via the table of contents. Use the verification badges to assess confidence level. Check revision dates before acting on tool-specific claims.

Table of Contents

Module 1: CLAUDE.md System Prompts — The 4-Purpose Framework

Module 2: Agent Harnesses — What Wraps the LLM

Module 3: Parallelization & Multi-Agent Patterns — Fan-Out, Consensus, Debate

Module 4: Auto Research — Karpathy's Autonomous Improvement Loop

Module 5: Browser & Computer Automation — HTTP to Full Computer Use

Module 6: Performance Diversification — Avoiding Monoculture Risk

Module 7: Multi-Agent Orchestration Landscape — Org Charts & Frameworks

Module 8: Workspace Organization — Business, Personal & Client Structures

Module 9: Security — The 80/20 That Actually Matters

Appendix A: Expert & Source Directory — 14 Verified Authorities

Appendix B: Tool & Repo Quick Reference — Links, Status, Star Counts

Appendix C: Implementation Checklists — Prioritized Action Items

Appendix D: Unverified & Personal Claims — What Could Not Be Confirmed

Module 1: CLAUDE.md System Prompts

Last verified: April 2, 2026 | Source: Anthropic docs (code.claude.com/docs/en/memory)

Core Concept: The 4-Purpose Framework

The course frames CLAUDE.md as serving four distinct purposes. This framework is a useful mental model, though not an official Anthropic categorization.

#	Purpose	What It Does	Where It Lives
1	**Knowledge Compression**	Summarizes workspace structure so Claude doesn't re-read every file each session. ~45x compression ratio claimed (827 words to 22 lines).	Local CLAUDE.md
2	**User Preferences**	Programming conventions, communication style, tool preferences, output formatting.	Global + Local
3	**Capability Declaration**	Tells Claude what it CAN do (APIs, tools, skills) so it doesn't underestimate its own abilities.	Local CLAUDE.md
4	**Log of Failures & Successes**	Running record of what worked and what didn't, shrinking the solution search space over time.	Local CLAUDE.md ("Lab Notes")

Scope Hierarchy VERIFIED

Course claims 2 scopes. Reality: 4+ scopes exist.

Scope	Location	Loaded When
Managed Policy	`/Library/Application Support/ClaudeCode/CLAUDE.md` (macOS)	Always (org-wide)
User / Global	`~/.claude/CLAUDE.md`	Every session, every workspace
Project	`./CLAUDE.md` or `./.claude/CLAUDE.md`	When in that project directory

Local (gitignored)	`./CLAUDE.local.md`	When in that project (not committed)
Subdirectory	`./subdir/CLAUDE.md`	Discovered on demand
Rules	`.claude/rules/*.md`	Path-scoped rules

Key Commands VERIFIED

Command	Function	Notes
`/init`	Scans workspace, generates starter CLAUDE.md	Interactive version behind `CLAUDE_CODE_NEW_INIT=1`
`/insights`	Analyzes last 30 days of sessions, generates HTML report	Uses Haiku for qualitative facets; can auto-generate rules
`/context`	Shows token usage breakdown	System prompt, MCP tools, memory, conversation, buffer

The Update Loop (Course Strategy)

```
LOCAL LOOP (frequent):                    GLOBAL LOOP (periodic):

  Plan Feature                               Accumulate local learnings
       |                                          |
  Instantiate (failures + successes)         Run /insights (30-day analysis)
       |                                          |
  Compile Learnings                          Manually review patterns
       |                                          |
  Update Local CLAUDE.md                     Add high-ROI bullets to Global
       |                                          |
  [repeat -- each cycle faster]              Update ~/.claude/CLAUDE.md
                                                  |
                                             [repeat after many local cycles]
```

What to Put Where

Global CLAUDE.md

- Context about you (goals, constraints, budget vs. time

Local CLAUDE.md

- Project structure summary (from `/init`)

preference)
- High-level reasoning strategies
- Global preferences (file path format, programming paradigm)
- Token conservation strategies
- Agency reminders ("you CAN do X")

- API docs and usage patterns
- Project-specific capabilities and skills
- Lab notes: failures, successes, things not to retry
- Deployment workflow (dev/build/preview/production)

Key Principle: Token Cost of Context

The course demonstrates a ~45x compression ratio. Full `app.jsx` = ~1,100 tokens. CLAUDE.md summary = ~22 tokens. This matters because:

- Token length scales inversely with output quality
- Every query pays the system prompt tax
- Opus 4.6 input: $5/MTok; Sonnet 4.6: $3/MTok -- real money at scale

Module 2: Agent Harnesses

Last verified: April 2, 2026 | Source: Anthropic engineering blog, LangChain

Definition VERIFIED

An **agent harness** is everything that wraps around the LLM that is not the LLM itself: the system prompt, hooks, available tools, permission modes, memory management, context compaction parameters, and execution environment.

LLM = intelligence. Harness = what makes it useful. Claude is the model. Claude Code is the harness.

Harness Components

Component	What It Controls
System Prompt	CLAUDE.md files, injected at top of context window
Tools	Bash, Read, Write, Edit, Grep, Glob, Web tools, MCP servers
Hooks	Lifecycle automation: PreToolUse, PostToolUse, SessionStart, etc.
Permission Modes	Plan mode, default, accept edits, auto mode, bypass permissions
Memory	Context compaction, autocompact buffer, conversation history
Parameters	Max turns, token limits, model selection

Primary Reference: Anthropic's Harness Blog VERIFIED

"Effective Harnesses for Long-Running Agents" -- Published Nov 26, 2025
anthropic.com/engineering/effective-harnesses-for-long-running-agents

Key concepts: initializer agent sets up environment (feature lists, git repos, progress tracking files like `claude-progress.txt`). Coding agents make incremental progress across multiple context windows. Each new window reads the progress file + git history.

LangChain's Harness Engineering Series VERIFIED

Post	Date	Key Insight

"Agent Frameworks, Runtimes, and Harnesses -- oh my!"	Nov 4, 2025	Taxonomy: frameworks vs. runtimes vs. harnesses
"Improving Deep Agents with Harness Engineering"	Feb 17, 2026	52.8 to 66.5 on Terminal-Bench by tweaking only the harness
"The Anatomy of an Agent Harness"	Mar 2026	Structural breakdown of harness components
"How Middleware Lets You Customize Your Agent Harness"	Mar 2026	Extensibility patterns

Alternative Harnesses Landscape

Harness	Status	Differentiator
Claude Code	DOMINANT	40.8% adoption. Best model + best harness integration
OpenAI Codex	VERIFIED	Strong backend/contemplation. Sandboxed cloud execution
Droid (Factory AI)	VERIFIED	58.75% Terminal-Bench. $50M funded. Enterprise focus
Pi (p.dev)	VERIFIED	Open-source, multi-provider. By Mario Zechner (badlogic)
Cursor	Active	IDE-integrated. Popular for frontend work

Auto Mode (Permission Innovation) VERIFIED

Launched: March 24, 2026 (research preview)
How it works: Sonnet 4.6 classifier reviews each action before execution. Blocks actions that escalate beyond task scope or appear driven by prompt injection.
Requires: Team, Enterprise, or API plan. Claude Sonnet 4.6 or Opus 4.6.
Enable: `--enable-auto-mode` at startup; cycle with `Shift+Tab`

Module 3: Parallelization & Multi-Agent Patterns

Last verified: April 2, 2026 | Sources: Microsoft Azure, MindStudio, OpenReview, Anthropic docs

Why Parallelize

Time

Serial: 4 steps x 5 min = 20 min. Parallel: 5 min concurrent + 2 min synthesis = 7 min. **65% faster.**

Quality

Model performance degrades as context grows. Sub-agents stay in the "zone of good" with short, clean context windows.

Coverage

Stochastic outputs: 5 runs produce ~2.5x unique answers vs. 1 run. Outlier ideas surface that a single agent would miss.

Cost

Research with Sonnet ($3/MTok), synthesize with Opus ($5/MTok). ~60% savings on research tokens.

Pattern 1: Fan-Out / Fan-In VERIFIED

```
                [Research Query]
                       |
        +------------+------------+
        |            |            |
[Sonnet Agent 1] [Sonnet 2] [Sonnet N]    <-- Fan Out (cheap, parallel)
        |            |            |
        +------------+------------+
                       |
              [Opus Synthesizer]             <-- Fan In (smart, single)
                       |
               [Final Answer]
```

Use when: Research tasks, API evaluation, competitor analysis, any "find the best X" query.

Documented by: Microsoft Azure Architecture Center, Kinde, multiple practitioners.

Pattern 2: Stochastic Consensus VERIFIED

```
Same prompt (slightly varied framing) --> N agents independently

Agent 1: [A, B, C]        Consensus count:
Agent 2: [B, C, E]        A: 2  B: 3  C: 3  (HIGH CONFIDENCE)
Agent 3: [A, B, Q]        E: 1  Q: 1        (OUTLIERS -- investigate)
```

Use when: Decision-making, ranking options, filtering hallucinations, surfacing high-variance ideas.

Documented by: MindStudio ("Stochastic Multi-Agent Consensus"), Kinde ("LLM Fan-Out 101").

Pattern 3: Multi-Round Debate VERIFIED

```
Time 1: Each agent generates independently
Time 2: Each agent sees ALL other responses, refines
Time 3: Further refinement with cross-pollination
...
Time N: Convergence on nuanced, battle-tested solutions
```

Use when: Nuanced analysis, architecture decisions, anything where "it depends" is the first answer.

Documented by: Edoardo Schepis (Medium series), academic papers on OpenReview.

Pattern 4: Pipeline (Sequential Specialist Handoff)

```
[Dev Agent] --> [QA Agent] --> [Test Agent]
     |              |              |
 Fresh context  Fresh context  Fresh context
 Incentive:     Incentive:     Incentive:
 Build fast     Find bugs      Break things
```

Use when: Any build-then-verify workflow. Different incentives prevent blind spots.

Recommended Orchestration Architectures

Parent + Researchers + QA (Recommended)

Parent (Opus): orchestrates, builds
Researchers (Sonnet x N): fan-out research
QA (Opus): fresh context, finds bugs
Best balance of quality, speed, cost.

Developer + QA (Lean)

Dev (Opus): plans and builds
QA (Opus): fresh spawn after each feature, zero context bias
Simpler, fewer compound probability issues.

Agent Teams Feature VERIFIED

Status: Experimental. Requires `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS` env var.
How it works: One session = team lead. Coordinates multiple independent teammates. Each has own context window. Can communicate via team lead.
Sub-agent model overrides: Supported via `model` field in SKILL.md frontmatter.

Module 4: Auto Research (Karpathy Framework)

Last verified: April 2, 2026 | Source: github.com/karpathy/autoresearch, Shopify/liquid PR #2056

The Three Prerequisites

Auto research works when -- and only when -- you have all three:

#	Requirement	Example (Website)	Example (Cold Email)
1	**Metric** to optimize	Google Lighthouse score	Reply rate %
2	**Change method** (fast)	Modify HTML/CSS/JS	Modify email copy
3	**Assessment** (fast)	Run Lighthouse test (~30s)	Wait for replies (~1hr)

Speed math: If change + assess = 60 seconds, you get 1,440 experiments/day. At 2% success rate with 1% improvement each = 1.01^30 = **34% improvement per day**.

The Loop VERIFIED

```
+---> [Hypothesize] ---> [Execute Change] ---> [Assess]
|                                                |
|          +--- improvement? ---> YES: Keep, log success
|          |                      NO:  Revert, log failure
|          |
+----------+

Research log accumulates: what worked, what didn't, why.
Each cycle starts with full history -- search space shrinks.
```

Verified Real-World Results

Who	What	Result	Source

Andrej Karpathy	GPU training pipeline optimization	~700 experiments, 20 improvements, 11% speedup on already-optimized code	VERIFIED github.com/karpathy/autoresearch
Toby Lutke (Shopify CEO)	Shopify Liquid Ruby template engine	53% faster parse+render, 61% fewer object allocations, ~120 experiments	VERIFIED Shopify/liquid PR #2056

Applicable Domains (Course Claims)

Domain	Metric	Change Method	Assessment	Loop Speed
Website speed	Lighthouse score	Modify code	Run Lighthouse	~60s/cycle
SaaS backend	Request latency	Modify server code	Benchmark endpoint	~60s/cycle
Conversion rate	Signup %	Modify landing page	A/B test results	~hours/cycle
Cold email	Reply rate	Modify copy/subject	Measure responses	~1hr/cycle
Ad creative	CTR / CPA	Modify creative	Platform metrics	~hours/cycle
Customer support	Resolution rate	Modify agent prompt	Track outcomes	~1hr/cycle

The Autonomy Spectrum

```
Vibe Coding          Agentic Engineering       Auto Research          ???
(2024-2025)          (2025-2026)               (2026+)               (Future)

Human: writes        Human: directs            Human: sets goal       Human: sets vision?
AI: codes            AI: orchestrates          AI: runs experiments
Role: writer         Role: director            Role: PI / researcher
```

Setup: 3 Files

File	Purpose
`program.md`	Instructions: what to modify, how to score, where to log
`train.py`	The thing being optimized (website, model, pipeline)
`progress.log`	Running log of all experiments, scores, and decisions

Module 5: Browser & Computer Automation

Last verified: April 2, 2026 | Source: Anthropic docs, browser-use.com

The Automation Spectrum

	HTTP Requests	Browser Automation	Computer Use
Speed	Fastest (~0.2s)	Medium (~5s/action)	Slowest (~10s/action)
Cost	Cheapest	Moderate	Most expensive (screenshots)
Setup	High (learn each API)	Low-medium	Zero
Generality	Narrow (one API at a time)	Broad	Universal (anything on screen)
Fragility	Very fragile	Moderate	Robust
Detection	Often blocked	Sometimes blocked	Rarely blocked
Tool	Claude's `WebFetch`	Chrome DevTools MCP, Browser Use	Claude Desktop Computer Use

Recommended Workflow (Course Strategy)

```
1. Prototype with Browser Automation (Chrome DevTools MCP)
   - General, works for most tasks
   - Can see network requests (hidden APIs)

2. If blocked --> Browser Use (browser-use.com)
   - Anti-detect fingerprinting
   - CAPTCHA solving, 195+ country proxies
   - Best for social media automation

3. Once flow is proven --> Convert to HTTP Requests
   - Extract API patterns from browser network tab
   - Document in workspace for future use
   - 100x faster, 100x cheaper at scale

4. If nothing else works --> Computer Use
```

```
    - Controls mouse + keyboard literally
    - Always works, extremely expensive
    - Last resort
```

Legal warning: Browser and computer automation may violate terms of service of target platforms. The course acknowledges this but does not condone violations. Understand the legal implications before automating interactions with third-party services.

Module 6: Performance Diversification

Last verified: April 2, 2026 | Source: conductor.build, GitHub repos

The Monoculture Risk

The course uses an "Interstellar blight" analogy: when one crop dominates, a single disease destroys everything. Claude Code at 40.8% adoption creates real dependency risk. Documented outages have caused widespread productivity drops.

Recommended Split: 70/30

```
Your Productivity Basket:

[Claude Code] [Claude Code] [Claude Code] [Claude Code]
[Claude Code] [Claude Code] [Claude Code]   = 70%

[Codex] [Codex]                              = 20%
[Pi / Local Models]                          = 10%
```

Three Diversification Methods VERIFIED

Method	Tool	When to Use	Status
Multi-agent platform	**Conductor** (conductor.build)	Daily work -- run Claude Code + Codex in parallel	VERIFIED YC-backed
MCP bridge	**Codex MCP Server** (tuannvm/codex-mcp-server)	Claude up but model degraded -- route tasks to Codex	VERIFIED
Full alternative	**OpenAI Codex desktop app**	Complete Claude outage -- switch platforms entirely	VERIFIED

Portability Strategy

Maintain parallel system prompt files:

- `CLAUDE.md` -- for Claude Code

- `agents.md` -- for Codex / provider-agnostic agents
- `gemini.md` -- for Gemini / VS Code agent

Add a meta-instruction to CLAUDE.md: "When you update this file, also sync changes to agents.md and gemini.md."

Module 7: Multi-Agent Orchestration Landscape

Last verified: April 3, 2026 | Source: GitHub repos verified individually

Key Insight: Skills and Sub-Agents Are the Same Thing

The course makes an important observation: SKILL.md files and sub-agent definitions share nearly identical structure (name, description, tools, prompt). The only real difference is context window state. This convergence suggests future unification.

Property	Sub-Agent	Skill
Fresh context	Yes	No (but compressed)
Specialized	Yes	Yes
Reliable at task	Yes	Yes
Markdown + tools	Yes	Yes
Delegates to others	Yes	Rarely

Verified Org Chart Projects

Project	Stars	Approach	Status
Paperclip (paperclipai/paperclip)	~45k	CEO/CTO/CMO agent roles, board oversight	VERIFIED
Gastown (steveyegge/gastown)	~13.4k	Mayor + Crew + Polecats + Refinery (by Steve Yegge)	VERIFIED
CrewAI (crewAIInc/crewAI)	~47.8k	Role-playing autonomous agents framework	VERIFIED
Open Goat (marian2js/opengoat)	~315	Hierarchical orgs across multiple agent platforms	VERIFIED
SwarmClaw (swarmclawai/swarmclaw)	~207	Agent runtime with orchestration/delegation	VERIFIED
Company Helm (CompanyHelm/companyhelm)	~7	AI studio with defined agent roles	VERIFIED (TINY)

Droid (Factory AI)	N/A	Commercial. CodeDroid, ReviewDroid, QA Droid	VERIFIED
"The System"	?	26 specialized agents (as described)	NOT FOUND

Course's own assessment: "I think pretty much all of them suck right now." The instructor views these as premature anthropomorphization of agents into human org structures. Simpler architectures (Parent + Researchers + QA) outperform complex hierarchies.

Module 8: Workspace Organization

Last verified: April 2, 2026 | Note: These are personal conventions, not official Anthropic recommendations

Core Structure

```
business/
  .env                    # API keys (gitignored)
  CLAUDE.md               # Local system prompt
  .claude/
    skills/               # Reusable SOPs
    settings.json         # Hooks, permissions
  active/                 # All generated output (never pollute root)
    tmp/                  # Ephemeral files (hidden)
    model-chat/           # Skill-specific output dirs
    research/
    web-design/
  clients/
    client-a/
      .env                # Client's API keys
      CLAUDE.md           # Client context (/init + manual)
      .claude/skills/     # Client-specific skills
    client-b/
      ...

personal/                 # Separate workspace, different VS Code color
  .env
  CLAUDE.md
  .claude/skills/
  active/
  health/                 # Domain-specific subfolders
  citizenship/
  ...
```

Key Principles

- **Never pollute root.** All generated files go in `active/` subdirectories.
- **Skills specify their output paths.** Each SKILL.md declares where its output lives.
- **Client folders mirror business structure.** Each has its own .env, CLAUDE.md, skills.

- **Periodically clean up.** Ask Claude to organize loose files in `active/`.
- **Color-code workspaces.** VS Code `.vscode/settings.json` with different title bar colors per workspace type.
- **Cross-workspace skill access.** Add a line to CLAUDE.md: "Client skills live in `clients/[name]/.claude/skills/` -- read from there when working on that client."

Alternative Approaches

- Project-based: `company/projects/{website, cro, lead-gen}`
- GitHub-synced: nightly push, daily clone, cloud backup
- CRM-as-JSON: store customer data in workspace, sync to calendar
- Or just ask Claude to design one for your needs

Module 9: Security -- The 80/20 That Actually Matters

Last verified: April 2, 2026 | Sources: OWASP, academic papers, Supabase docs

Guiding Principle

"Everything is hackable. It's always just a question of how hackable." The goal is not perfection -- it's making your attack surface more expensive to breach than the value of what's behind it. A fence and a camera, not Fort Knox.

The Five Low-Hanging Threats

1. Plain-Text API Keys in Conversation Logs VERIFIED

Risk: All Claude Code conversations stored at `~/.claude/projects/` in JSONL format. Any API key pasted in chat = stored in plain text on disk.
Fix: Store ALL secrets in `.env`. Reference by variable name only. Never paste raw keys into chat. Add `.env` to `.gitignore`.

2. Hallucinated Package Names (Supply Chain) VERIFIED

Risk: AI models hallucinate package names ~19.7% of the time (academic study: "We Have a Package for You," 2024). Attackers register these misspelled names with malware.
Fix: Audit dependency list after every `npm install`. Ask Claude: "Are all packages here legitimate, verified, and actively maintained?" Check for unfamiliar names before running.

3. Database Row-Level Security (RLS) VERIFIED

Risk: Supabase does NOT enable RLS by default. Without it, any user with a public key can read, write, and delete every row in your database.
Fix: Enable RLS on every table. Immediately. No exceptions.

4. Public-Facing Agent Servers

Risk: Any URL on a VPS IP range is continuously scanned by bot farms testing for vulnerabilities. Open ports, unpatched services, and exposed agent interfaces are found within hours.
Fix: Never expose agent interfaces directly. Use authenticated proxies (Telegram connectors, etc.). Don't give public agents access to sensitive data (passports, SSNs, credentials).

5. Credit Card Data

Risk: Storing card numbers anywhere = PCI compliance nightmare + massive liability. AI agents will log them in conversation history.
Fix: Use Stripe or equivalent payment processor. Never touch card numbers. Period.

Security Audit Workflow

1. Build your feature / app / system
2. Open a **new, clean conversation** (no development context bias)
3. Paste a security audit prompt covering: secret scanning, dependency audit, RLS check, input validation, OWASP Top 10 for LLMs
4. Review findings, then open **another new conversation** to implement fixes (avoids audit bias)
5. Optionally: run the audit with a different model (Codex, Gemini) for provider diversity

Appendix A: Expert & Source Directory

Last verified: April 3, 2026 | These people are NOT the course instructor -- they independently document the same concepts

Boris Cherny
Anthropic Staff Engineer. Created Claude Code. 259 AI-written PRs in 30 days.

Matt Pocock
"Claude Code for Real Engineers" cohort course. TypeScript educator.

Cole Medin (coleam00)
AI Agents Masterclass (YouTube). Created your-claude-engineer harness. Conference speaker.

Steve Yegge
Gastown creator (~13.4k stars). Veteran engineer. Famous for "Google Platforms Rant."

Theo (t3dotgg)
YouTuber (~500K subs). Extensive Claude Code coverage and tutorials.

Mario Zechner (badlogic)
Pi agent creator (open-source, multi-provider). Technical blog at mariozechner.at.

Andrej Karpathy
Auto-research framework creator. Former Tesla head of AI. OpenAI co-founder.

Simon Willison
Covers AI tools extensively at simonwillison.net. Detailed auto-research analysis.

Florian Bruniaux
Created claude-code-ultimate-guide on GitHub.

LangChain Team
4+ blog posts on harness engineering. Taxonomy of frameworks vs. runtimes vs. harnesses.

Microsoft Azure Team
"AI Agent Design Patterns" reference architecture. Fan-out/fan-in documentation.

Anthropic (Official)
code.claude.com/docs. Engineering blog. Coursera: "Claude Code: Software Engineering with AI Agents."

ClaudeLog (claudelog.com)
Community documentation aggregator. Tutorials, guides, tips.

Scrimba
Published "Best Claude Code Tutorials and Courses in 2026" curated roundup.

Appendix B: Tool & Repo Quick Reference

Last verified: April 3, 2026 | Star counts and status change rapidly -- re-verify before citing

Claude Code Features

Feature	Status	Access
CLAUDE.md (4+ scopes)	STABLE	Built-in
`/init`	STABLE	Built-in (new interactive: `CLAUDE_CODE_NEW_INIT=1`)
`/insights`	STABLE	Built-in (analyzes 30-day history)
`/context`	STABLE	Built-in
Skills (SKILL.md)	STABLE	`.claude/skills/` + anthropics/skills repo
Hooks	STABLE	`.claude/settings.json`
Agent Teams	EXPERIMENTAL	`CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS`
Sub-agent model overrides	STABLE	`model` field in SKILL.md frontmatter
Auto Mode	RESEARCH PREVIEW	Team/Enterprise/API plan. `--enable-auto-mode`

External Tools & Platforms

Tool	What	Link	Verified
Karpathy Auto Research	Autonomous improvement loop	github.com/karpathy/autoresearch	YES
Conductor	Multi-agent parallel runner (Mac)	conductor.build	YES
Codex MCP Server	Bridge Claude Code to OpenAI Codex	github.com/tuannvm/codex-mcp-server	YES

Browser Use	Anti-detect browser automation	browser-use.com	YES
Chrome DevTools MCP	Browser automation via DevTools	Various MCP server repos	YES
Pi Agent	Open-source multi-provider harness	github.com/badlogic/pi-mono	YES
Droid (Factory AI)	Enterprise agent suite	factory.ai	YES

Multi-Agent Frameworks

Project	GitHub	Stars	Verified
CrewAI	crewAIInc/crewAI	~47.8k	YES
Paperclip	paperclipai/paperclip	~45k	YES
Gastown	steveyegge/gastown	~13.4k	YES
Open Goat	marian2js/opengoat	~315	YES
SwarmClaw	swarmclawai/swarmclaw	~207	YES
Company Helm	CompanyHelm/companyhelm	~7	YES (TINY)
"The System"	Unknown	?	NOT FOUND

Appendix C: Implementation Checklists

Prioritized by verified impact. Check off as you implement.

Tier 1: High-Value, Verified, Well-Documented

- ☐ Set up global CLAUDE.md at `~/.claude/CLAUDE.md` with: personal context, reasoning preferences, token conservation rules, agency reminders
- ☐ Run `/init` on every active project to generate local CLAUDE.md
- ☐ Run `/insights` monthly to surface patterns and auto-generate rules
- ☐ Store ALL API keys in `.env` only -- add `.env` to `.gitignore`
- ☐ Enable Supabase RLS on every table in every project
- ☐ Implement fan-out/fan-in pattern: Sonnet researchers + Opus synthesizer
- ☐ Set up auto-research loop on one metric you care about (website speed is easiest starter)
- ☐ Run security audit prompt on any public-facing project (use fresh conversation, no dev context)
- ☐ Add "Lab Notes" section to local CLAUDE.md -- log failures and successes after each development cycle

Tier 2: Medium-Value, Situational

- ☐ Use stochastic consensus for important decisions (spawn N agents, aggregate by frequency)
- ☐ Use debate pattern for nuanced architectural decisions
- ☐ Install Conductor for multi-provider parallel agent work
- ☐ Set up Codex MCP server as Claude Code fallback
- ☐ Configure Browser Use for any social media or anti-bot automation needs
- ☐ Create SKILL.md files for any SOP you repeat more than 3 times
- ☐ Separate business and personal workspaces with distinct VS Code color themes
- ☐ Structure client folders to mirror business workspace (.env, CLAUDE.md, skills)

Tier 3: Nice-to-Have

- ☐ Maintain provider-portable system prompts (sync CLAUDE.md to agents.md, gemini.md)
- ☐ Schedule periodic workspace cleanup ("organize loose files in active/")
- ☐ Audit npm dependencies after every AI-driven install for hallucinated package names
- ☐ Try Agent Teams feature (experimental) for complex multi-part tasks
- ☐ Explore auto-research on non-code metrics: email copy, ad creative, customer support prompts

Appendix D: Unverified & Personal Claims

These claims could not be independently confirmed via GitHub or public sources

Claim	Status	Notes
"Over $4 million a year in profit"	UNVERIFIABLE	No public financial records. Common influencer claim.
"Teach around 2,000 people"	PLAUSIBLE	Consistent with community/course size, but not independently confirmed.
"350,000 YouTube subscribers"	CHECKABLE	Verify on YouTube channel "Nyx Drive" -- subscriber counts are public.
"Uses Claude Code daily in business"	PLAUSIBLE	Demonstrated working knowledge throughout. Consistent with claims.
"Owns multiple companies including LeftClick"	PARTIAL	leftclick.ai exists and is a real agency. Other companies not verified.
"The System" (26 agents project)	NOT FOUND	May be private, misnamed, or misheard in audio transcription.

Editorial Assessment

The instructor demonstrates genuine, deep expertise with Claude Code. His technical claims are overwhelmingly accurate (30+ verified against primary sources). The strategies he presents -- CLAUDE.md management, parallelization patterns, auto-research, workspace organization, security 80/20 -- are **well-documented by multiple independent sources** including Anthropic themselves, LangChain, Microsoft, and academic researchers.

He is a skilled practitioner and effective communicator. He is **not the sole authority** on any of these topics. At least 14 other credible voices independently document the same concepts. Use this binder as a cross-referenced resource, not a single-source dependency.

End of Binder

Claude Code Advanced Practitioner's Binder | Rev 1.1 | April 3, 2026
Compiled by Prompt AI Solutions (promptaisolutions.com)

This document is a point-in-time snapshot. The AI tooling landscape changes weekly. Re-verify all tool availability, feature flags, pricing, and star counts before acting. Target shelf life: 60-90 days. Schedule re-verification for June 2026.

No single source. No single expert. Cross-reference everything.